

CodeCheck: архитектура платформы проверки заданий

CodeCheck принимает решения студентов, проверяет их в изолированной песочнице и возвращает результат; три микросервиса на Python общаются через Kafka.

Ключевые решения

Выбор по каждому пункту задания собран в одной таблице; обоснования расписаны в разделах ниже.

Область	Решение	Почему
Домен	Платформа проверки заданий по программированию	Тяжёлая асинхронная обработка естественно требует очереди и событий
Сервисы	<code>courses-service</code> (Flask), <code>submissions-service</code> (FastAPI), <code>grader-service</code> (FastAPI)	У каждого свой владелец данных и своя скорость изменений
Шина событий	Kafka (режим KRaft)	Replay для перепроверки решений, несколько независимых групп потребителей
Реляционные данные	PostgreSQL	Пользователи, курсы, задания, метаданные решений: нужны транзакции и связи
NoSQL	MongoDB	Исходники и логи запусков: документы разной формы, TTL для «холодных» логов
Кэш	Valkey (форк Redis, тот же протокол)	Тесты заданий, рейтинг, ключи идемпотентности
Слой роутинга	Kong (DB-less)	Маршрутизация, балансировка и rate limiting в одном компоненте
Согласованность	Transactional outbox + идемпотентные консьюмеры	Нет двойной записи в БД и Kafka, доставка at-least-once
Песочница	Одноразовый Docker-контейнер: без сети, лимиты CPU, RAM и времени	Изоляция недоверенного кода студентов

Область	Решение	Почему
Запуск	Docker Compose	Вся система поднимается одной командой, достаточно для запуска и проверки

Техническое задание

ТЗ составлено по ключевым пунктам ГОСТ 34.602-89 (автоматизированная система) с учётом структуры ГОСТ 19.201-78 (программное изделие); для учебного проекта они объединены в один документ.

1. Общие сведения

- Наименование: автоматизированная система проверки учебных заданий по программированию «CodeCheck».
- Исполнитель: один разработчик; заказчик: преподаватель курса.

2. Назначение и цели

Система принимает решения студентов, автоматически проверяет их запуском тестов в изоляции и выдаёт результат и рейтинг. Цели измеримы:

- результат проверки типового решения за 30 с и быстрее (p95) при очереди до 100 решений;
- ни одно принятое решение не теряется, даже если проверяющий сервис недоступен (до 7 дней, затем перепроверка из БД);
- преподаватель сам добавляет задание с тестами, без участия разработчика.

3. Характеристика объекта автоматизации

Учебные курсы по программированию на Python. Допущение для расчёта нагрузки: около 2 000 студентов и 20 преподавателей, пики отправок перед дедлайном (до 300 решений за 5 минут, то есть около 1 решения в секунду). Нагрузочный прогон проводится с запасом до 5 RPS.

4. Требования к системе

4.1. Функциональные требования

№	Требование
FR-1	Регистрация и вход, роли student, teacher, admin, аутентификация по JWT
FR-2	Преподаватель создаёт курс и задание: набор тестов (stdin и ожидаемый stdout), лимиты времени и памяти

№	Требование
FR-3	Студент записывается на курс и видит его задания
FR-4	Студент отправляет решение (файл на Python, до 64 КБ) и сразу получает номер решения со статусом <code>queued</code>
FR-5	Система проверяет решение в песочнице и присваивает вердикт: <code>accepted</code> , <code>wrong_answer</code> , <code>runtime_error</code> , <code>time_limit_exceeded</code> , <code>memory_limit_exceeded</code> , <code>system_error</code>
FR-6	Студент видит статус и результат по тестам; для скрытых тестов показывается только вердикт
FR-7	Система ведёт рейтинг студентов по заданию
FR-8	Преподаватель запускает перепроверку всех решений задания после изменения тестов
FR-9	Повторная отправка с тем же <code>Idempotency-Key</code> не создаёт дубликат решения
FR-10	Преподаватель видит список решений студентов по заданию и их вердикты
FR-11	Администратор видит состояние очереди и сервисов: метрики Prometheus (лаг консьюмера) и health-эндпоинты

4.2. Нефункциональные требования. Приведены с числами в разделе «User stories, use cases и NFR».

4.3. Требования к видам обеспечения

Вид	Требование
Программное	Python 3.12, Flask, FastAPI, PostgreSQL, MongoDB, Kafka, Valkey, Kong, Docker
Информационное	Контракты API в OpenAPI 3, схемы событий в JSON Schema с версией
Техническое	Для демонстрации достаточно одной машины: 4 CPU, 8 ГБ RAM, Docker Compose
Организационное	Один разработчик, код в Git, решения по

Вид	Требование
	архитектуре фиксируются в ADR

5. План работ (по мотивам стадий ГОСТ 34.601-90)

Этап	Содержание	Результат
1. Архитектура	ТЗ, требования, C4, sequence, выбор технологий	Этот документ (стадии: формирование требований, концепция, ТЗ, эскизный проект)
2. Каркас	Compose-стенд, Kong, Kafka, базы, health-эндпоинты	Стенд поднимается одной командой (технический проект)
3. Сервисы	API трёх сервисов, outbox, события	Сквозной сценарий FR-1...FR-4, FR-6, FR-7, FR-9 (рабочая документация и разработка)
4. Проверка	Песочница, лимиты, перепроверка	Сценарии FR-5, FR-8, FR-10, FR-11
5. Качество	Тесты, нагрузочный прогон, проверка отказов	Отчёт по NFR (испытания)
6. Сдача	Документация, Postman-коллекция, демонстрация	Готовый пакет (ввод в действие)

6. Порядок контроля и приёмки

Приёмка идёт по сценариям UC-01...UC-06 и требованиям FR-10, FR-11 через Postman-коллекцию и включает три проверки отказоустойчивости:

- остановить `grader-service`, отправить решения, запустить сервис обратно: все решения проверены, дублей нет;
- превысить лимит отправок: клиент получает HTTP 429;
- нагрузочный прогон на пиковый профиль из NFR: целевые p95 выдержаны.

7. Требования к документированию

OpenAPI-описание каждого сервиса, README с запуском, диаграммы C4 и sequence, каталог событий Kafka, ADR для ключевых решений (шина событий, шлюз, хранилища).

8. Источники разработки

ГОСТ 34.602-89, ГОСТ 34.601-90, ГОСТ 19.201-78.

User stories, use cases и NFR

Три роли (студент, преподаватель, администратор), шесть сценариев и двенадцать нефункциональных требований; все числа целевые и опираются на допущение о нагрузке из ТЗ.

User stories

№	Роль	Хочу	Чтобы
US-1	Студент	отправить решение и сразу получить подтверждение приёма	не ждать проверку с открытой страницей
US-2	Студент	видеть статус и результат по каждому тесту	понять, что исправлять
US-3	Студент	видеть рейтинг по заданию	сравнивать себя с группой
US-4	Студент	повторить отправку после сбоя сети без дубля	не засорять историю попыток
US-5	Преподаватель	создать задание с тестами и лимитами	запускать проверку без разработчика
US-6	Преподаватель	перепроверить все решения после правки тестов	исправить ошибку в условиях задним числом
US-7	Преподаватель	видеть решения группы по заданию	оценивать прогресс
US-8	Администратор	видеть состояние очереди и сервисов	заметить перегрузку до жалоб студентов

Use cases

№	Актор и цель	Основной сценарий	Альтернативы и ошибки
UC-01	Любой пользователь: войти	Логин и пароль → courses - service проверяет → выдаёт access и refresh JWT	Неверный пароль: 401; превышен лимит попыток: 429
UC-02	Преподаватель:	Запрос на создание →	Нет прав: 403; тесты

№	Актор и цель	Основной сценарий	Альтернативы и ошибки
	создать задание	валидация тестов и лимитов → запись в PostgreSQL → событие <code>assignment.updated</code>	пусты: 422
UC-03	Студент: отправить решение	Запрос с файлом и <code>Idempotency-Key</code> → запись решения и события в одной транзакции → ответ 202 со статусом <code>queued</code> → проверка → событие с вердиктом → статус обновлён	Повтор ключа: вернуть прежнее решение; задание закрыто: 409; файл больше лимита: 413; превышен лимит отправок: 429
UC-04	Студент: посмотреть результат	Запрос статуса → кэш → при промахе PostgreSQL → ответ с вердиктом и разбором тестов	Чужое решение: 404; проверка ещё не завершена: статус <code>queued</code> (в очереди или выполняется)
UC-05	Преподаватель: перепроверить задание	Запрос → <code>submissions-service</code> повторно публикует события по всем решениям задания → <code>grader-service</code> проверяет по новым тестам → вердикты обновляются	Задание без решений: 200 с нулём в счётчике; повторный запрос во время перепроверки: 409
UC-06	Студент: посмотреть рейтинг	Запрос → чтение из Valkey (sorted set) → список лучших и место пользователя	Кэш пуст: пересборка из PostgreSQL

Нефункциональные требования

№	Категория	Требование
NFR-1	Производительность	POST /submissions отвечает за 300 мс и

№	Категория	Требование
		быстрее (p95) при пике около 1 RPS (запас до 5 RPS), ответ не ждёт проверки
NFR-2	Производительность	GET статуса: 150 мс и быстрее (p95), при попадании в кэш 50 мс и быстрее
NFR-3	Время проверки	Результат за 30 с и быстрее (p95) для типового решения (до 4 тестов по 0,5 с) при очереди до 100; лимит на запуск задаёт преподаватель, не более 10 с CPU и 256 МБ RAM
NFR-4	Доступность	Целевая доступность API 99,5 % в месяц (в учебном стенде с одним экземпляром сервисов и одним брокером не гарантируется); при остановке grader-service приём решений продолжается
NFR-5	Сохранность	Принятое решение не теряется: решение и событие записываются в PostgreSQL одной транзакцией до ответа клиенту; публикация в Kafka (acks=all) идёт после ответа через outbox-relay с повторами
NFR-6	Семантика доставки	At-least-once; дубли гасит идемпотентный обработчик; на пару (решение, версия тестов) один итоговый вердикт, при перепроверке (FR-8) вердикт обновляется
NFR-7	Масштабируемость	grader-service масштабируется горизонтально до числа партиций топика (6)

№	Категория	Требование
NFR-8	Безопасность	JWT: access 15 минут, refresh 7 дней; RBAC по ролям; чужой код запускается только в песочнице без сети, от непривилегированного пользователя, с read-only файловой системой
NFR-9	Ограничение нагрузки	10 отправок в минуту на пользователя (точный лимит считает submissions-service, Kong ограничивает грубо по токену); 100 запросов в минуту на IP; вход и регистрация 5 в минуту на IP; ответ 429 с Retry-After. Студенты за одним NAT делят квоту по IP, это известное ограничение стенда
NFR-10	Наблюдаемость	JSON-логи с correlation_id, метрики Prometheus (включая лаг консьюмера), /health/live и /health/ready
NFR-11	Хранение	Исходники 1 год, логи запусков 90 дней (TTL-индекс), события в Kafka 7 дней; если проверка недоступна дольше 7 дней, события восстанавливаются перепроверкой из PostgreSQL (FR-8)
NFR-12	Сопровождаемость	Покрытие тестами ядра логики не ниже 70 %, OpenAPI у каждого сервиса

Микросервисы и события

Система делится на три сервиса по владению данными: каждый пишет только в свои хранилища, а изменения состояния сообщает событиями в Kafka. Kong и инфраструктура (Kafka, базы, Valkey) в число микросервисов не входят.

Сервисы

Сервис	Стек	Отвечает за	Владеет данными	Публикует	Читает
<code>courses-service</code>	Flask	Пользователи и JWT, курсы, записи на курс, задания, тесты	PostgreSQL <code>courses_db</code>	<code>assignment.updated</code>	не читает
<code>submissions-service</code>	FastAPI	Приём решений, статусы, вердикты, рейтинг, перепроверка	PostgreSQL <code>submissions_db</code> , MongoDB <code>sources</code> , рейтинг в Valkey	<code>submission.created</code>	<code>submission.graded</code>
<code>grader-service</code>	FastAPI и воркер	Запуск решения в песочнице, вердикт, логи запуска	MongoDB <code>run_logs</code> (TTL), кэш тестов в Valkey	<code>submission.graded</code>	<code>submission.created</code> , <code>assignment.updated</code>

Синхронных вызовов между сервисами два, оба на чтение и оба с кэшем: `submissions-service` и `grader-service` запрашивают у `courses-service` данные задания (срок сдачи, тесты, лимиты). Если `courses-service` недоступен, оба работают по кэшу.

Каталог событий

Топик	Ключ	Партиции	Кто пишет	Кто читает	Зачем
<code>assignment.updated</code>	<code>assignment_id</code>	3	<code>courses-service</code>	<code>grader-service</code>	Сбросить кэш тестов, запомнить <code>tests_version</code>
<code>submission.created</code>	<code>submission_id</code>	6	<code>submissions-service</code>	<code>grader-service</code> (группа <code>grader</code>)	Запрос на проверку, внутри исходник до 64 КБ
<code>submission.created.retry</code>	<code>submission_id</code>	6	<code>grader-service</code>	<code>grader-service</code>	Повтор после временной ошибки с

Топик	Ключ	Партиции	Кто пишет	Кто читает	Зачем
					задержкой
submission.created.dlq	submission_id	1	grader-service	человек	Решения, не проверенные за 3 попытки
submission.graded	submission_id	6	grader-service	submissions-service	Вердикт и результаты по тестам

У каждого события одинаковый конверт: `event_id`, `type`, `schema_version`, `occurred_at`, `correlation_id`, `payload`. Исходник кладётся прямо в событие, потому что лимит 64 КБ (иначе ответ 413) избавляет `grader-service` от обратного вызова в `submissions-service`. Для больших файлов потребовался бы паттерн «claim check» с хранением в объектном хранилище.

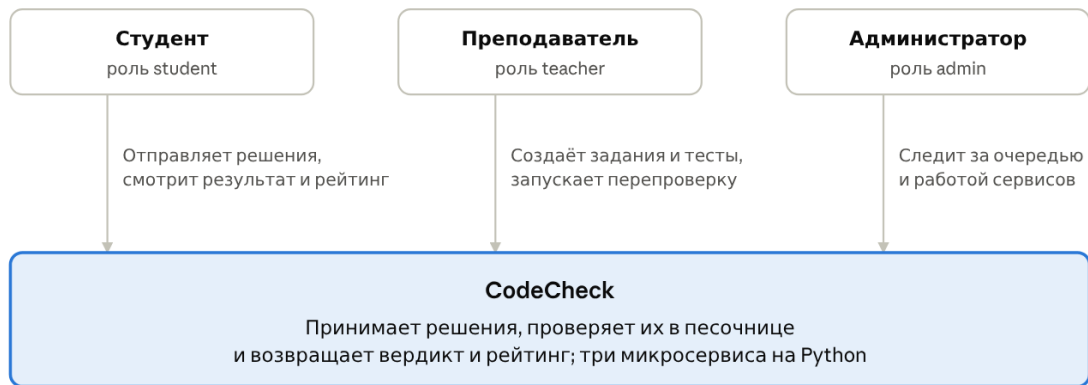
Правила согласованности

- **Transactional outbox.** `submissions-service` и `courses-service` пишут строку данных и строку события в одной транзакции PostgreSQL; отдельный процесс публикует события в Kafka и помечает их отправленными. Так нет рассинхрона между БД и брокером.
- **Идемпотентный результат.** `grader-service` коммитит offset только после публикации `submission.graded`. При сбое проверка может повториться, поэтому `submissions-service` хранит результат с уникальным ключом (`submission_id`, `tests_version`) и игнорирует дубли и устаревшие версии тестов.
- **Повторы и DLQ.** Временная ошибка (песочница недоступна) отправляет сообщение в `retry`; после 3 попыток оно уходит в `dlq`, а решение получает вердикт `system_error`.
- **Долгие задачи.** Проверка занимает до десятков секунд, поэтому `max.poll.interval.ms` задаётся с запасом над максимальным временем запуска, а потребитель не опрашивает брокер на время работы.

C4, уровень 1: контекст

CodeCheck — одна система с тремя ролями пользователей и без внешних интеграций.

Три роли работают с одной системой CodeCheck

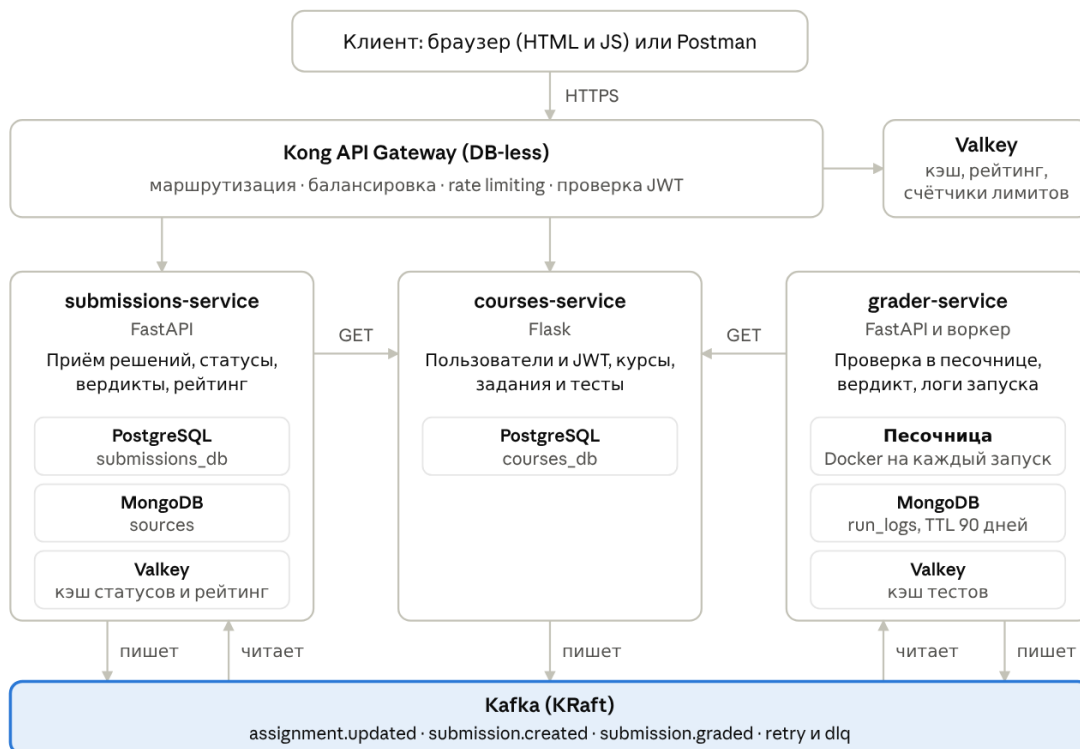


C4 L1 · 3 роли, 1 система

C4, уровень 2: контейнеры

Три сервиса обмениваются событиями через Kafka и ходят друг к другу по HTTP только за чтением данных задания.

Сервисы общаются через Kafka, синхронно только читают задания



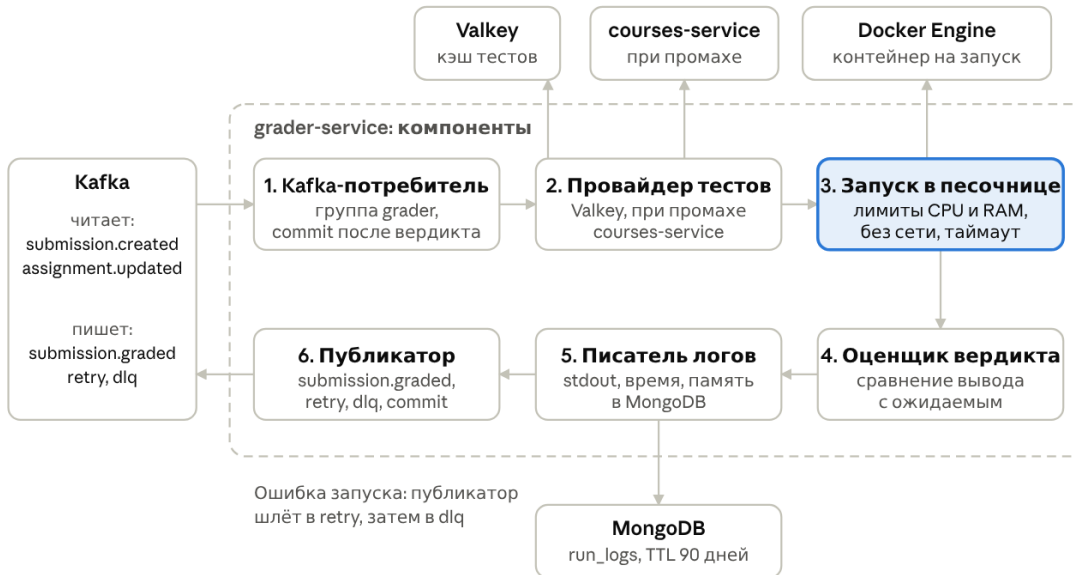
C4 L2 · 3 сервиса, Kafka, Kong, хранилища

Внутри рамки сервиса показаны хранилища, которыми он владеет; Valkey в Compose один общий экземпляр, ключи разделены префиксами. У Kong нет маршрута к grader-service.

C4, уровень 3: компоненты grader-service

Запуск в песочнице вынесен в отдельный компонент, а порядок обработки одного сообщения задан номерами 1–6: оценщик вердикта не знает про запуск, а запуск не знает про Kafka.

Проверка решения идёт конвейером из шести компонентов

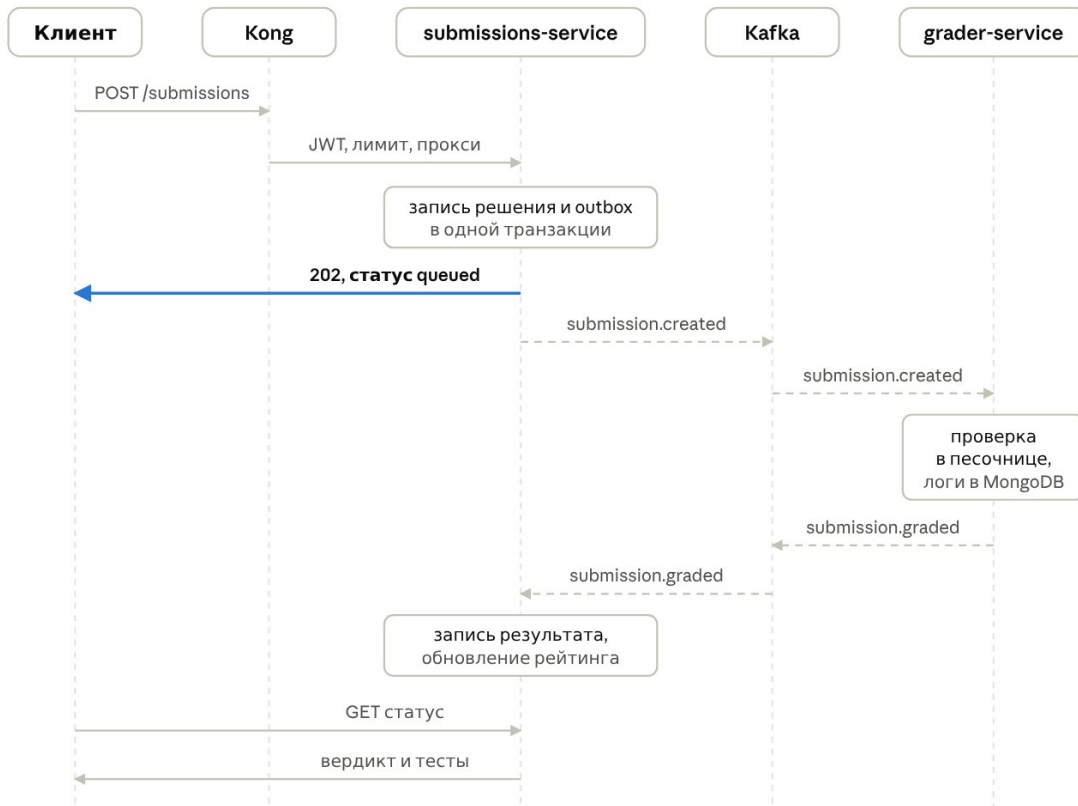


C4 L3 · 6 компонентов grader-service

Sequence: отправка и проверка решения

Клиент получает ответ 202 сразу после записи решения в базу, а проверка и обновление статуса идут асинхронно, поэтому время ответа не зависит от длины очереди.

Клиент получает 202 сразу, а проверка идёт асинхронно через Kafka



Сплошная линия: HTTP, пунктир: событие в Kafka

sequence · UC-03 и UC-04, 5 участников

Kafka, RabbitMQ и NATS: выбор шины

Выбрана Kafka, но честный вывод такой: для текущей нагрузки (пик около 1 RPS, одна очередь задач) объективно лучше подошёл бы RabbitMQ, на втором месте NATS JetStream. Kafka оправдана требованием задания и тем, что вокруг потока решений легко вырастают новые потребители.

Сравнение

Критерий	Kafka	RabbitMQ	NATS (JetStream)
Модель	Распределённый журнал: топика, партиции, группы потребителей, pull	Брокер очередей (AMQP): обменники, очереди, push	Лёгкий pub/sub, JetStream добавляет хранение и потребителей
Хранение	По сроку и размеру, независимо от прочтения	Сообщение удаляется после ack; журнал даёт только режим	Хранится в стриме по политике

Критерий	Kafka	RabbitMQ	NATS (JetStream)
		Streams	
Replay	Перемотка offset в любой момент в окне retention	Только в Streams	Есть, по номеру или времени
Порядок	Внутри партиции	Внутри очереди, нарушается при нескольких потребителях и requeue	Внутри стрима
Ack, повторы, DLQ	Offset на группу; повторы и DLQ строятся вручную (retry-топики)	Ack на сообщение, nack, requeue, dead-letter exchange, TTL встроены	Ack wait, лимит доставок, переотправка встроены
Параллелизм потребителей	Не больше числа партиций в группе	Конкурирующие потребители без жёсткого лимита	Queue groups и pull-потребители, лимита нет
Пропускная способность	Очень высокая	Умеренная, десятки тысяч сообщений в секунду на узел	Очень высокая в core, ниже в JetStream
Эксплуатация	Тяжелее: брокеры, KRaft-кворум, планирование партиций	Средняя: один узел для учебных целей	Минимальная: один бинарник
Python-клиенты	aiokafka, confluent-kafka	aio-pika, pika	nats-py

Почему всё же Kafka

- **Требование задания.** Шина через Kafka входит в обязательный скоуп.
- **Несколько независимых читателей одного потока.** `submission.graded` сегодня читает один сервис; уведомления, антиплагиат и аналитика подключаются отдельными группами без правок продюсера.
- **Журнал как аудит.** События хранятся 7 дней, их можно перечитать, чтобы пересобрать рейтинг или найти причину спорного вердикта.
- **Порядок по ключу.** События одного решения идут в одну партицию, поэтому обрабатываются по порядку.

Цена выбора

- Повторы и DLQ делаются руками через топики `retry` и `dlq`; в RabbitMQ это настройка очереди.
- Параллелизм проверки ограничен числом партиций (6), RabbitMQ масштабировал бы воркеров свободно.
- Брокер тяжелее в эксплуатации; в Compose это один узел без репликации, что допустимо для демонстрации, но не для продакшена.
- Перепроверка (UC-05) не опирается на replay: события заново публикуются из PostgreSQL, потому что окно retention короче жизни решений.

Гибрид (RabbitMQ для очереди проверки и Kafka для потока событий) не берём: два брокера для одного разработчика дают больше сложности, чем пользы.

EDA в проекте

- **Хореография без оркестратора.** Сервисы реагируют на события друг друга; саги не нужны, потому что распределённой транзакции нет: цепочка из двух шагов (принять, проверить) с компенсацией через `system_error`.
- **Итоговая согласованность.** После приёма решение сразу видно со статусом `queued` (запись в свою БД до ответа), вердикт появляется позже; клиент опрашивает `GET` статуса, позже можно добавить SSE.
- **Контракты событий версионизируются** полем `schema_version`; потребитель игнорирует неизвестные поля.

Данные

PostgreSQL хранит то, что связано ключами и требует транзакций; MongoDB хранит документы разной формы и с ограниченным сроком жизни, включая «холодные» логи запусков. Внешних ключей между базами разных сервисов нет: на чужие сущности ссылаются обычным идентификатором.

Хранилища

Хранилище	Владелец	Что лежит	Почему здесь	Срок
PostgreSQL <code>courses_db</code>	<code>courses-service</code>	Пользователи, курсы, записи, задания, тесты, outbox	Связи, ограничения, транзакции	Бессрочно
PostgreSQL <code>submissions_db</code>	<code>submissions-service</code>	Решения, результаты, outbox	Уникальные ключи для идемпотентности, выборки по пользователю и	Бессрочно

Хранилище	Владелец	Что лежит	Почему здесь	Срок
			заданию	
MongoDB sources	submissions- service	Исходный код решения	Документ читается целиком, связей нет, строки PostgreSQL остаются лёгкими	1 год
MongoDB run_logs	grader- service	Вывод, время и память по каждому тесту	Форма данных разная, объём растёт быстро, нужен автоматический TTL	90 дней
Valkey	все три	Кэш, рейтинг, ключи идемпотентност и	См. раздел о кэше	От минут до недель

В Compose это один экземпляр PostgreSQL с двумя базами и один MongoDB с двумя базами, у каждого сервиса свой пользователь и доступ только к своей базе. В продакшене базы разносятся по инстансам.

Основные сущности

База	Сущность	Ключевые поля и ограничения
courses_db	users	id, email (уникален), password_hash, role, created_at
courses_db	courses	id, title, owner_id → users
courses_db	enrollments	course_id, user_id, составной первичный ключ
courses_db	assignments	id, course_id, title, deadline, time_limit_ms, memory_limit_mb, tests_version, is_open
courses_db	test_cases	id, assignment_id, stdin, expected_stdout,

База	Сущность	Ключевые поля и ограничения
		is_hidden, position
submissions_db	submissions	id, user_id, assignment_id, status, idempotency_key, tests_version, created_at; уникальны (user_id, idempotency_key)
submissions_db	results	submission_id, tests_version, verdict, tests_passed, tests_total, time_ms, graded_at; первичный ключ (submission_id, tests_version)
обе базы	outbox	id, topic, key, payload (JSONB), created_at, sent_at
sources	документ	_id = номер решения, code, size_bytes, created_at
run_logs	документ	submission_id, tests_version, test_index, stdout, stderr, time_ms, memory_kb, exit_code, created_at; TTL-индекс на created_at (90 дней)

Индексы под основные запросы: submissions (user_id, assignment_id, created_at DESC) для истории попыток и results (verdict) для статистики преподавателя.

Кэширование

Кэш на Valkey работает по схеме cache-aside: сервис читает ключ, при промахе идёт в источник и кладёт результат с TTL. Valkey — открытый форк Redis с тем же протоколом, поэтому клиент redis-пу подходит без изменений.

Что кэшируем

Ключ	Что хранит	Кто использует	TTL	Инвалидация
<code>assignment: {id}</code>	Карточка задания: дедлайн, лимиты, <code>tests_version</code> , <code>is_open</code>	<code>submissions- service</code>	60 с	Только по TTL
<code>tests: {assignment_id}: {tests_version}</code>	Набор тестов задания	<code>grader- service</code>	1 ч	Версия входит в ключ, новая версия не пересекается со старой; <code>assignment.updated</code> удаляет ключи прошлых версий
<code>status: {submission_id}</code>	Статус и вердикт решения	<code>submissions- service</code>	3 с для промежуточных статусов, 10 мин для итогового	Удаляется при записи результата
<code>lb: {assignment_id}</code>	Рейтинг: sorted set, участник <code>user_id</code> , балл	<code>submissions- service</code>	Без TTL	Обновляется при <code>submission.graded</code> ; при потере ключа пересобирается из PostgreSQL
<code>idem: {user_id}: {key}</code>	Номер решения для Idempotency-Key	<code>submissions- service</code>	24 ч	Только по TTL; истина хранится в уникальном ключе PostgreSQL

Счётчики rate limiter ведёт Kong в том же Valkey со своим префиксом; их описание в разделе о слое роутинга.

Правила

- **Политика вытеснения `volatile-lru`.** При нехватке памяти удаляются только ключи с TTL, рейтинг без TTL остаётся. Лимит памяти для демонстрации 256 МБ.
- **Деградация без падения.** Если Valkey недоступен, сервисы читают из PostgreSQL и `courses-service`: медленнее, но работают.

- **Известный компромисс.** Решение, принятое в первые 60 с после правки тестов, может получить старый `tests_version` из кэша карточки и проверится по старым тестам. Исправляется перепроверкой (FR-8).
- **Кэш не источник истины.** Всё, что лежит в Valkey, восстанавливается из баз; поэтому персистентность для него не включается.

Слой роутинга

Входная точка системы — Kong в режиме DB-less: один компонент закрывает три роли задания (маршрутизатор, балансировщик, rate limiter), а конфигурация лежит в репозитории файлом `kong.yml`. Публичных маршрутов к `grader-service` нет, он доступен только изнутри сети.

Роли и их реализация

Роль	Как реализована	Настройки
Маршрутизатор	Services и routes по префиксу пути; плагин <code>jwt</code> проверяет подпись токена, плагин <code>correlation-id</code> проставляет сквозной идентификатор запроса	<code>/api/auth</code> , <code>/api/courses</code> , <code>/api/assignments</code> ведут в <code>courses-service</code> ; <code>/api/submissions</code> , <code>/api/leaderboard</code> ведут в <code>submissions-service</code>
Балансировщик	Upstream на сервис с несколькими targets (реплики контейнеров)	<code>least-connections</code> для <code>submissions-service</code> , <code>round-robin</code> для остальных; активные health checks по <code>/health/ready</code> и пассивные по ошибкам и таймаутам; connect timeout 2 с, read timeout 10 с; один повтор на другой реплике
Rate limiter	Плагин <code>rate-limiting</code> с политикой <code>redis</code> , фиксированное окно, счётчики в Valkey	См. таблицу лимитов ниже

Повтор запроса на другой реплике безопасен и для `POST /submissions`, потому что повтор с тем же Idempotency-Key не создаёт второго решения.

Лимиты

Маршрут	Ключ лимита	Лимит	Зачем
<code>POST</code>	IP	5 в минуту	Защита от подбора пароля

Маршрут	Ключ лимита	Лимит	Зачем
/api/auth/login			
POST /api/submissions	Токен (Kong) и пользователь из JWT (сервис)	10 в минуту	Защита очереди проверки (NFR-9)
Остальные маршруты	IP	100 в минуту	Общая защита (NFR-9)

При превышении клиент получает HTTP 429 с заголовками лимита.

Решения и компромиссы

- **Фиксированное окно** допускает до двух пачек запросов на стыке окон. Для учебной нагрузки это приемлемо; скользящее окно или token bucket потребовали бы самописного лимитера на Lua или в ASGI-слое либо платной версии плагина.
- **Fail-open.** Если Valkey недоступен, плагин с `fault_tolerant` пропускает запросы без лимита: доступность важнее, а перегрузку проверки сдерживает очередь в Kafka.
- **Почему Kong, а не альтернативы.** NGINX с `limit_req` и Traefik в открытой версии считают лимиты на каждом экземпляре отдельно, а общие счётчики в Redis требуют доработки или платной версии; Kong делает это штатно.
- **Админ-интерфейс** Kong не публикуется, конфигурация меняется только через репозиторий.

Соответствие реализации

Код лежит в архиве проекта (README.md описывает запуск); в таблице всё, что в нём отличается от текста выше.

Пункт документа	Как сделано в коде	Причина
Лимит 10 отправок в минуту на пользователя	Kong ограничивает по IP (100 в минуту, вход 5) и по токену (10 отправок); точный лимит на пользователя считает <code>submissions-service</code> в Valkey, повтор с тем же <code>Idempotency-Key</code> квоту не тратит	Kong OSS не умеет считать лимит по <code>claim</code> из JWT
Статус <code>running</code>	Нет: <code>queued</code> , затем вердикт	Начало проверки не публикуется событием
Рейтинг по курсу	Только по заданию: балл равен проценту пройденных тестов, берётся лучшая	Сокращён скоуп

Пункт документа	Как сделано в коде	Причина
	попытка	
Пауза партии на время проверки	Нет: сообщение обрабатывается целиком, <code>max.poll.interval.ms</code> равен 10 минутам	Один поток на consumer, опрос брокера не нужен между шагами
Повторы и DLQ	Топик <code>submission.created.retry</code> , задержка 5 с на номер попытки, три попытки, затем DLQ и вердикт <code>system_error</code>	Как в документе
<code>results</code>	Добавлена колонка <code>details</code> (JSON по тестам), статус читается без MongoDB	Меньше обращений к хранилищам
<code>memory_kb</code> в <code>run_logs</code>	Не заполняется, лимит памяти проверяется по OOM контейнера	CLI Docker не отдаёт пик памяти
OpenAPI у каждого сервиса	<code>/docs</code> у FastAPI-сервисов; для Flask описание API в README и коллекции Postman	Нет генератора схемы во Flask
MongoDB с отдельными пользователями	Один экземпляр без аутентификации, роли по сервисам только в PostgreSQL	Упрощение стенда
Схемы БД	Создаются при старте через <code>create_all</code> , миграций нет	Alembic в списке доработок